

ROBOT EMBEDDED PROGRAMING

Lab 6 Coordinate Frame Management Using tf



Name:

Lance Quan

Michael McCorkell

Percy Sanford

Tyler Brannon

HUMBER POLYTECHNIC

FACULTY OF APPLIED SCIENCES AND TECHNOLOGY

Objective

The objective of this lab is to introduce students to coordinate frame management in ROS 2 Jazzy using the tf2 system. Students will build a URDF model of a simple robot, publish its state through a Python node, and broadcast the robot's transforms. The lab also emphasizes visualization, where students will use RViz2 to observe the robot geometry, motion, and transforms. Through this work, students gain hands-on experience with URDF modeling, tf transformations, and visualization tools in ROS 2.

Learning outcomes

By the end of this lab, students will be able to:

1. Import an external robot 3D CAD geometry into ROS 2 and integrate it with a URDF model.
2. Create a minimal URDF file to define the geometry and configuration of a robot.
3. Develop a ROS 2 Jazzy package and launch file to load and visualize a CAD-based robot model.
4. Publish robot states using a Python node and broadcast transforms through the tf2 system.
5. Visualize robot geometry and transformations in RViz2, including coordinate frames and robot motion.
6. Demonstrate understanding of the role of URDF, launch files, and tf in managing robot description and visualization within ROS 2.



Source

Prerequisites

Source the ROS2 environment by typing this:

source /opt/ros/jazzy/setup.bash

printenv ROS_DISTRO

```
mystic@Zallmos:~$ source /opt/ros/jazzy/setup.bash
mystic@Zallmos:~$ printenv ROS_DISTRO
jazzy
```

Make sure we we have all necessary ROS2 packages.

We need **joint_state_publisher_gui** which enables us to set the joint position and orientation and to publish the **JointState** message. This is a package that creates a Graphics User Interface (GUI) for setting and publishing joint state values for a given URDF. Then, we will need the **robot_state_publisher** package for the robot_state_publisher node. We will also need RVIz package. This package is used to visualize the geometry and motion of a robot by using a simple GUI.

source /opt/ros/jazzy/setup.bash

sudo apt install ros-jazzy-joint-state-publisher-gui

```
mystic@Zallmos:~$ sudo apt install ros-jazzy-joint-state-publisher-gui
[sudo] password for mystic:
Sorry, try again.
[sudo] password for mystic:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ros-jazzy-joint-state-publisher-gui is already the newest version (2.4.3-1noble.20260912.164400).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

sudo apt install ros-jazzy-robot-state-publisher

```
mystic@Zallmos:~$ sudo apt install ros-jazzy-robot-state-publisher
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ros-jazzy-robot-state-publisher is already the newest version (3.3.4-1noble.20260915.184559).
ros-jazzy-robot-state-publisher set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

sudo apt install ros-jazzy-rviz2

```
mystic@Zallmos:~$ sudo apt install ros-jazzy-rviz2
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ros-jazzy-rviz2 is already the newest version (14.1.24-1noble.20260915.204213).
ros-jazzy-rviz2 set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

ros2 pkg list

ros2 pkg list | grep joint

```
mystic@Zallmos:~$ ros2 pkg list | grep joint
joint_limits
joint_state_publisher
joint_state_publisher_gui
rqt_joint_trajectory_controller
```

joint_state_publisher
joint_state_publisher_gui

ros2 pkg list | grep rviz

```
mystic@Zallmos:~$ ros2 pkg list | grep rviz
rviz2
rviz_assimp_vendor
rviz_common
rviz_default_plugins
rviz_ogre_vendor
rviz_rendering
```

rviz2
rviz_assimp_vendor
rviz_common
rviz_default_plugins
rviz_ogre_vendor
rviz_rendering

Tip: If you would like to navigate to the main folder, open new file explorer on windows (if you are using WSL2) and type:

\\wsl$\Ubuntu-24.04\home\<your-username>

Instructions:

Create the Package

Open a terminal, source the ROS2 underlay, and create a workspace folder

source /opt/ros/jazzy/setup.bash

cd ~
mkdir -p lab6_ws/src

```
mystic@Zallmos:~$ cd ~
mystic@Zallmos:~$ mkdir -p lab6_ws/src
```

You have two options for creating a package: using a C++ build system or a Python build system. In this lab, we will use the C++ build system—not to write C++ code, but simply to generate the appropriate source file structure. To begin, navigate to the **source** folder:

cd ~/lab6_ws/src

```
mystic@Zallmos:~$ cd ~/lab6_ws/src
```

Create an empty package

ros2 pkg create --build-type ament_cmake demo_robot

```
mystic@Zallmos:~/lab6_ws/src$ ros2 pkg create --build-type ament_cmake demo_robot
going to create a new package
package name: demo_robot
destination directory: /home/mystic/lab6_ws/src
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['mystic <mystic@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_cmake
dependencies: []
creating folder ./demo_robot
creating ./demo_robot/package.xml
creating source and include folder
creating folder ./demo_robot/src
creating folder ./demo_robot/include/demo_robot
creating ./demo_robot/CMakeLists.txt

[WARNING]: Unknown license 'TODO: License declaration'.  This has been set in the package.xml, but no LICENSE file has b
een created.
It is recommended to use one of the ament license identifiers:
Apache-2.0
BSL-1.0
BSD-2.0
BSD-2-Clause
BSD-3-Clause
GPL-3.0-only
LGPL-3.0-only
MIT
MIT-0
```

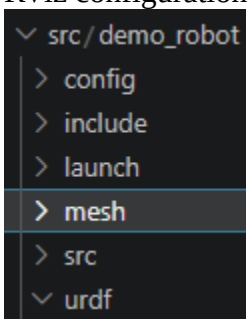
The package is named **“demo_robot”**, which generates the standard ROS 2 package file structure.

cd ~/lab6_ws

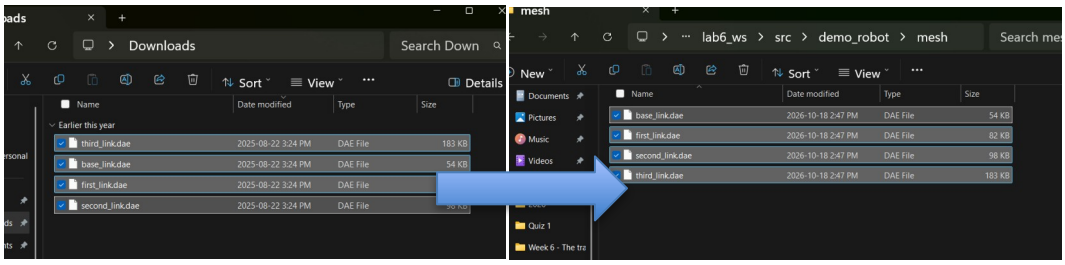
```
mystic@Zallmos:~/lab6_ws/src$ cd ~/lab6_ws
mystic@Zallmos:~/lab6_ws$ code .
```

Next, open VS (code .) and navigate to the package’s base folder
/lab6_ws/src/demo_robot

Next, let us create four folders (**launch, urdf, config, mesh)** inside of this folder. The folder "**launch**" will be used to store the Python launch file. The folder "**urdf**" will be used to store the "**urdf**" file of the model. The "**config**" folder will be used to store the RViz configuration file. The folder **mesh** will be used to store the CAD geometry.



We usually export every part of the robot as a Collada (dae) file. Download the files provided on BB, navigate via file explorer to mesh folder, and save the downloaded files in the mesh folder. Save files as **base_link.dae**, **first_link.dae**, **second_link.dae**, and **third_link.dae**.



Back to VS, navigate to the "urdf" folder and create a URDF file. Call the urdf file "**model.urdf**". Copy pastes the content of the model.urdf file provided (on BB)

The next step is to create a launch file. Navigate to the folder "launch" (the full path is "**~/demo_ws/src/demo_robot/launch**"). Inside of that folder create a file called "**display.launch.py".** Copy pastes the content of the launch file provided (on BB)

Modify the package.xml and CMakeLists.txt Files

Before we can build the package, we need to modify two files: package.xml and CMakeLists.txt files. These two files are located in the base package folder "**~/demo_ws/src/demo_robot**"

Open the file called **package.xml**, and add these lines:

<exec_depend>joint_state_publisher_gui</exec_depend>
<exec_depend>robot_state_publisher</exec_depend>

<exec_depend>rviz2</exec_depend>

This will tell to the ROS2 build system that our package depends on these additional packages. The next step is to modify the CMake file of the complete project. Open the file called CMakeLists.txt and add the following lines:

```
install(
  DIRECTORY launch urdf config mesh
  DESTINATION share/${PROJECT_NAME}
)
```

before “if(BUILD_TESTING)”. These lines will ensure that the CMake system can locate the launch, urdf, and rviz configuration files.

Build the project

To build the project. Go to the base folder of the project:

```
cd ~/lab6_ws
```

Before we build the project, we need to make sure that all the dependencies are installed on the system. To do that, we use **rosdep**. To install rosdep, type this

sudo apt-get install python3-rosdep

```
mystic@Zallmos:~/lab6_ws$ cd ~/lab6_ws
mystic@Zallmos:~/lab6_ws$ sudo apt-get install python3-rosdep
[sudo] password for mystic:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  python3-rosdep-modules
The following NEW packages will be installed:
  python3-rosdep python3-rosdep-modules
0 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
Need to get 58.6 kB of archives.
After this operation, 370 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://packages.ros.org/ros2/ubuntu noble/main amd64 python3-rosdep-modules all 0.27.0-1 [55.0 kB]
Get:2 http://packages.ros.org/ros2/ubuntu noble/main amd64 python3-rosdep all 0.27.0-1 [3,560 B]
Fetched 58.6 kB in 0s (117 kB/s)
Selecting previously unselected package python3-rosdep-modules.
(Reading database ... 153242 files and directories currently installed.)
Preparing to unpack .../python3-rosdep-modules_0.27.0-1_all.deb ...
Unpacking python3-rosdep-modules (0.27.0-1) ...
Selecting previously unselected package python3-rosdep.
Preparing to unpack .../python3-rosdep_0.27.0-1_all.deb ...
Unpacking python3-rosdep (0.27.0-1) ...
Setting up python3-rosdep-modules (0.27.0-1) ...
Setting up python3-rosdep (0.27.0-1) ...
```

Then, type this

sudo rosdep init
rosdep update

```
mystic@Zallmos:~/lab6_ws$ sudo rosdep init
Wrote /etc/ros/rosdep/sources.list.d/20-default.list
Recommended: please run

        rosdep update

mystic@Zallmos:~/lab6_ws$ rosdep update
```

Verify that all ROS2 dependencies are installed by running this

rosdep install -i --from-path src --rosdistro jazzy -y

```
mystic@Zallmos:~/lab6_ws$ rosdep install -i --from-path src --rosdistro jazzy -y
#All required rosdeps installed successfully
```

Finally, we can build everything by executing this command in the base workspace folder **~/lab6_ws/**

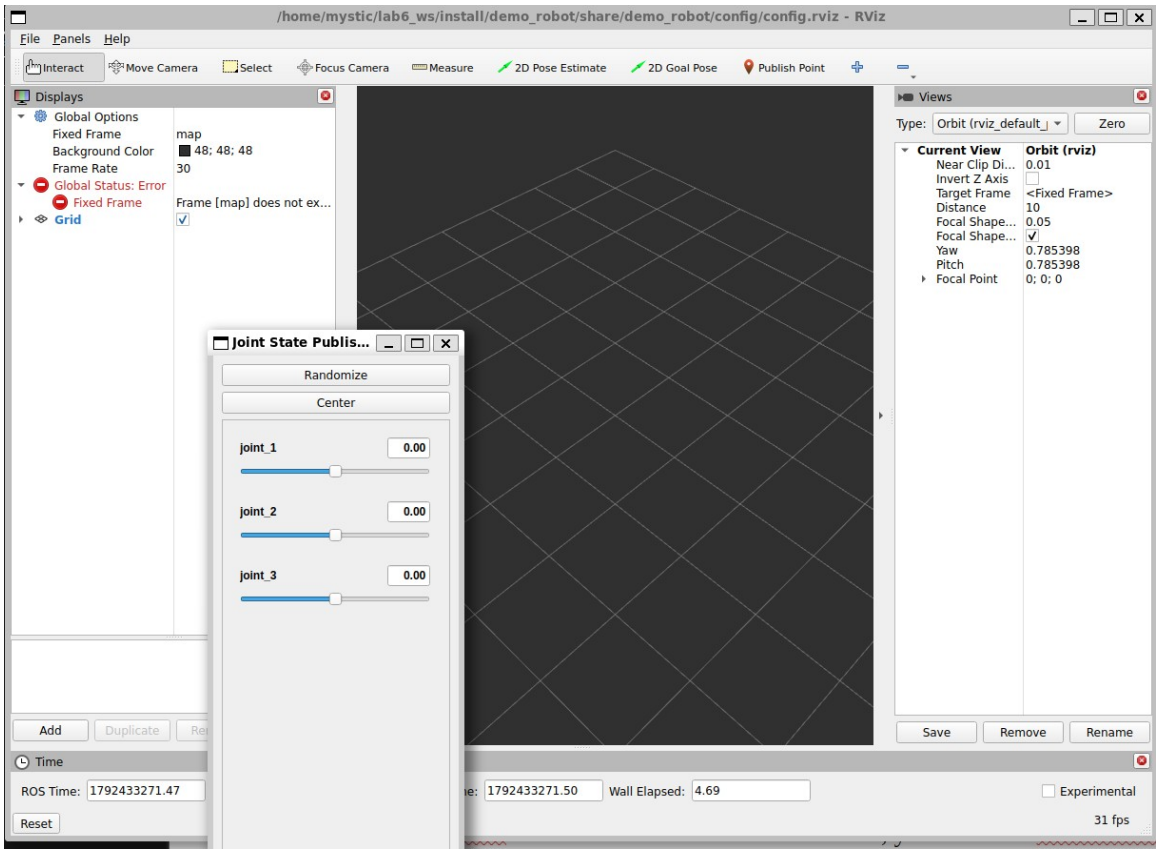
colcon build --symlink-install --packages-select demo_robot

Next, let us start the launch file. To do that, open a new terminal, and create an overlay by sourcing the environment:

source ~/lab6_ws/install/setup.bash

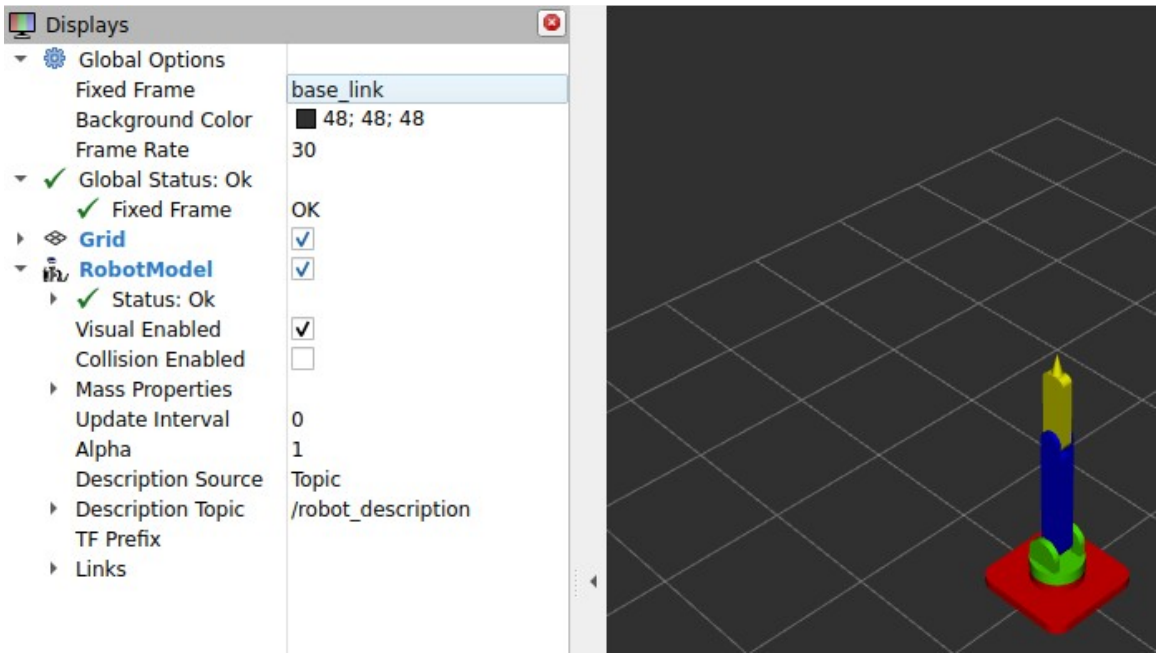
Finally, run the launch file by typing this

ros2 launch demo_robot display.launch.py



At first, nothing will appear in the RViz window. This happens because the parameters need to be adjusted manually for the model to display correctly. After making the adjustments, save the configuration to a file. The next time you open RViz, the saved parameters will load automatically, and no further manual adjustments will be needed.

Here is how to adjust the RViz parameters.
1.) Click on “Add” in Display menu, and click on “By display type” tab. Then, search for “RobotModel”. Select this and click on OK. Then, you will see “RobotModel” in “Displays” menu on the left. Expand the “RobotModel”. Find “Description Source”. From the drop-down menu next to “Description Source”, select “Topic”. Then, find under “RobotModel”, the option “Description Topic”, and from the drop-down menu select “/robot_description”. You will see that links of the robot will appear on the screen. However, they are not ordered. To order them, under “Global Options” (in “Displays” menu), find “Fixed Frame”, and from the drop-down menu select “base_link”. Now, you will observe that the complete robot model is properly displayed.



2.) Save the current configuration in the file. Click on “File”, “Save Config As”. The name of the file should be “config.rviz”. The file should be saved in the created **config** folder.

Test the saved parameters. Close RViz. Then, we need to rebuild the package. Open a new terminal and type

source /opt/ros/jazzy/setup.bash
cd ~/lab6_ws/
colcon build

```
mystic@Zallmos:~$ source /opt/ros/jazzy/setup.bash
mystic@Zallmos:~$ cd ~/lab6_ws
mystic@Zallmos:~/lab6_ws$ colcon build
Starting >>> demo_robot
Finished <<< demo_robot [0.35s]

Summary: 1 package finished [0.43s]
```

Then, open a new terminal
source /opt/ros/jazzy/setup.bash
source ~/lab6_ws/install/setup.bash
ros2 launch demo_robot display.launch.py

```
mystic@Zallmos:~$ source /opt/ros/jazzy/setup.bash
mystic@Zallmos:~$ source ~/lab6_ws/install/setup.bash
mystic@Zallmos:~$ ros2 launch demo_robot display.launch.py
[INFO] [launch]: All log files can be found below /home/mystic/.ros/log/2026-10-19-14-18-16-620887-Zallmos
[INFO] [launch]: Default logging verbosity is set to INFO
/home/mystic/lab6_ws/install/demo_robot/share/demo_robot/urdf/model.urdf
[INFO] [robot_state_publisher-1]: process started with pid [3756]
[INFO] [joint_state_publisher_gui-2]: process started with pid [3757]
[INFO] [rviz2-3]: process started with pid [3758]
[robot_state_publisher-1] [INFO] [1792433896.709827844] [robot_state_publisher]: Robot initialized
[rviz2-3] [INFO] [1792433896.865608047] [rviz2]: Stereo is NOT SUPPORTED
[rviz2-3] [INFO] [1792433896.865778883] [rviz2]: OpenGl version: 4.5 (GLSL 4.5)
[rviz2-3] [INFO] [1792433896.906171577] [rviz2]: Stereo is NOT SUPPORTED
[joint_state_publisher_gui-2] [INFO] [1792433897.117131364] [joint_state_publisher_gui]: Got description,
bot
[joint_state_publisher_gui-2] [INFO] [1792433897.135914264] [joint_state_publisher_gui]: Centering
[joint_state_publisher_gui-2] [INFO] [1792433897.246154365] [joint_state_publisher_gui]: Got description,
bot
[joint_state_publisher_gui-2] [INFO] [1792433897.249550581] [joint_state_publisher_gui]: Got description,
bot
[joint_state_publisher_gui-2] [INFO] [1792433897.252013658] [joint_state_publisher_gui]: Centering
[joint_state_publisher_gui-2] [INFO] [1792433897.255344161] [joint_state_publisher_gui]: Centering
```

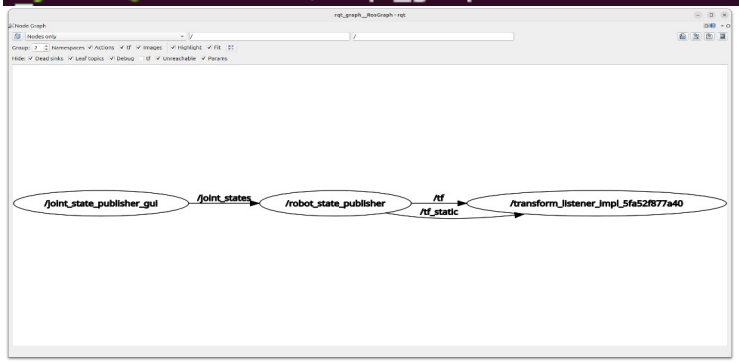
This will display everything.

Nodes and topics

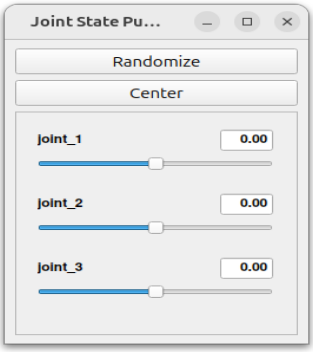
This graph is displayed by running these commands in a new terminal (after building everything)

sudo apt update && sudo apt update
sudo apt install '~nros-jazzy-rqt*'
source /opt/ros/jazzy/setup.bash
rqt_graph

```
mystic@Zallmos:~$ source /opt/ros/jazzy/setup.bash
mystic@Zallmos:~$ rqt_graph
```



This graph shows the nodes and the communication topics between them. The first node is **/joint_state_publisher_gui**, which opens a graphical interface for adjusting the robot's joint positions and orientations. This interface appears as the slider window used in the RViz simulation:



When the user sets the position and orientation of a joint, its state is published on the **/joint_states** topic.

In ROS 2, the state of a joint (whether revolute or prismatic) includes three components:

1. **Position**
    - Revolute joints: angle of rotation, in radians (rad).
    - Prismatic joints: linear displacement, in meters (m).
2. **Velocity**
    - Revolute joints: angular velocity, in radians per second (rad/s).
    - Prismatic joints: linear velocity, in meters per second (m/s).
3. **Effort**
    - Revolute joints: applied torque, in Newton-meters (Nm).
    - Prismatic joints: applied force, in Newtons (N).

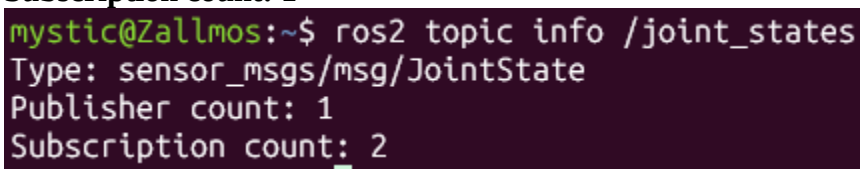
Note: This definition of “state” differs from the kinematic definition used in state-space models, where only positions and velocities are considered as states, and forces/torques are treated as inputs.

The joint state information is published and received through the **/joint_states** topic. To view details about this topic, run the following command (after sourcing your ROS 2 environment):

ros2 topic info /joint_states

The output should look somehow like this

Type: sensor_msgs/msg/JointState
Publisher count: 1
Subscription count: 1



Here, the most important type is the type of message being sent, and this is **sensor_msgs/msg/JointState.** This is the message that contains the state. To verify this, type this

ros2 interface show sensor_msgs/msg/JointState

```
mystic@Zallmos:~$ ros2 interface show sensor_msgs/msg/JointState
# This is a message that holds data to describe the state of a set of torque controlled joints.
#
# The state of each joint (revolute or prismatic) is defined by:
#  * the position of the joint (rad or m),
#  * the velocity of the joint (rad/s or m/s) and
#  * the effort that is applied in the joint (Nm or N).
#
# Each joint is uniquely identified by its name
# The header specifies the time at which the joint states were recorded. All the joint states
# in one message have to be recorded at the same time.
#
# This message consists of a multiple arrays, one for each part of the joint state.
# The goal is to make each of the fields optional. When e.g. your joints have no
# effort associated with them, you can leave the effort array empty.
#
# All arrays in this message should have the same size, or be empty.
# This is the only way to uniquely associate the joint name with the correct
# states.

std_msgs/Header header
        builtin_interfaces/Time stamp
                int32 sec
                uint32 nanosec
        string frame_id

string[] name
float64[] position
float64[] velocity
float64[] effort
```

The output looks like this

This is a message that holds data to describe the state of a set of torque controlled joints……
#.....
The state of
string[] name
float64[] position
float64[] velocity
float64[] effort

The **/robot_state_publisher** node receives joint state information and, using the robot's URDF model, calculates the 3D poses of each link based on the robot's kinematic tree. It relies on the robot_description parameter (URDF) and the joint positions from the **/joint_states** topic to perform forward kinematics. The resulting transformations are then published to the **/tf** topic.
The **tf2** library manages these transformations, maintaining a tree of coordinate frames and allowing vectors, points, and other quantities to be transformed between frames over time.

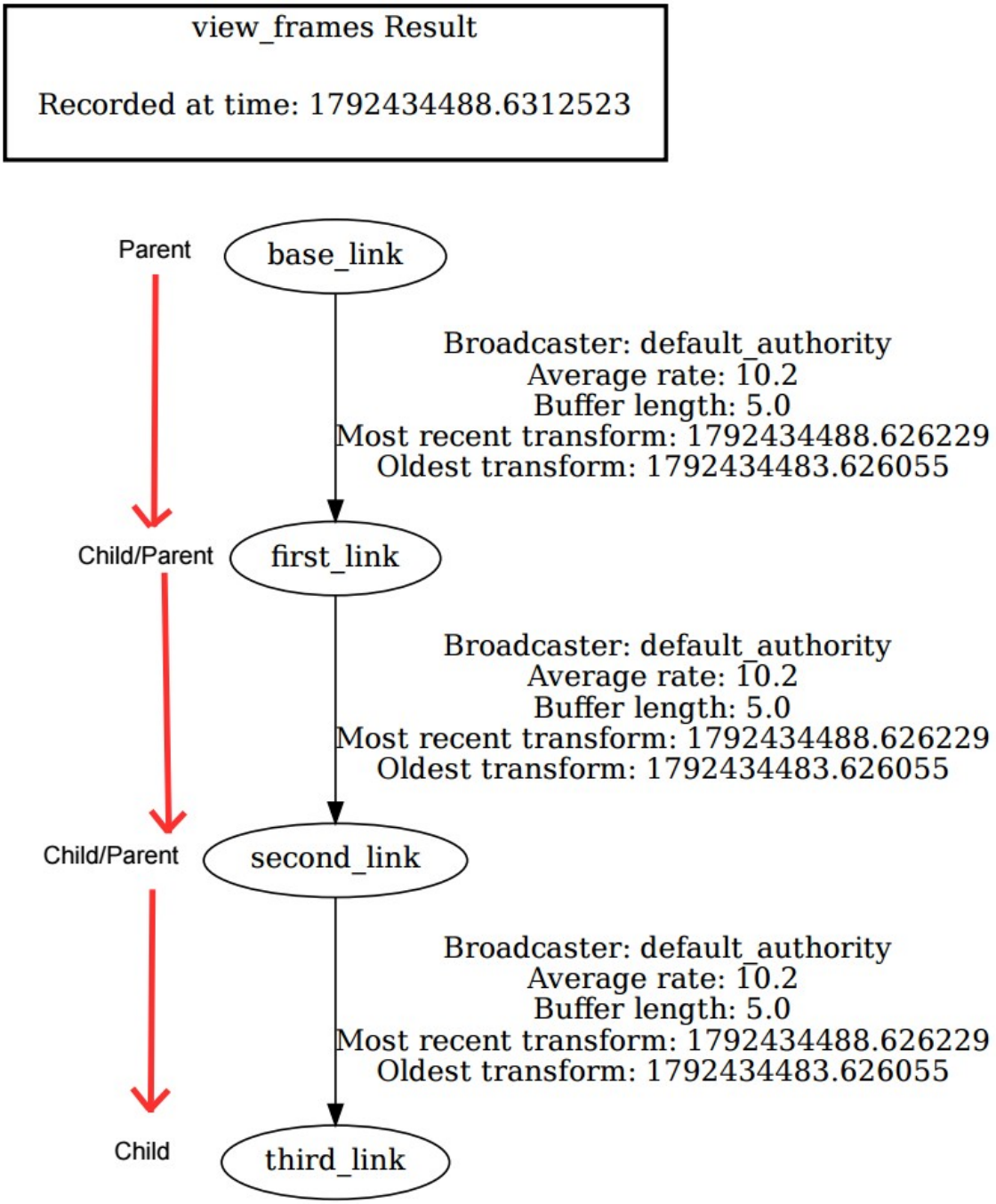
The transformations published to **/tf** are then received by the **/transform_listener** node. This makes it possible to visualize the robot's motion and coordinate frames in **RViz**. Although RViz runs as a node, it does not appear in the rqt_graph.

Deliverables

By the end of this lab, students must submit:

<u>Short Report</u> (1–2 pages)

1. TF Tree



Joint 1:
parent link: base_link
child link: first_link

Joint 2:
parent link: first_link
child link: second_link

Joint 3:
parent link: second_link
child link: third_link

2. Short Answer Questions

1. Why does RViz show the links in the wrong place until the Fixed Frame is set to `base_link`?
RViz places the links incorrectly until the Fixed Frame is set to base_link because the Fixed Frame is the reference that every other frame's position is calculated from. base_link is the root of the transform tree (the parent of the child links), so setting it as the Fixed Frame lets RViz compute each child's pose relative to it. If you chose a different link as the Fixed Frame instead, that link would become the reference, and everything would be positioned relative to it.

2. What would happen to the TF tree if one joint were missing from the URDF?
The processed image showed a "No tf data received" message, meaning RViz wasn't getting any transform data. Without it, RViz couldn't build the TF tree, so it couldn't position the links and the model was drawn incorrectly.

3. If `/joint_state_publisher_gui` is stopped, what happens to the transforms of the moving links? Why?
The model stays frozen in place until a transform is published for the link. For example, running ros2 run tf2_ros static_transform_publisher 0 0 0 1.57 0 0 base_link third_link publishes a fixed transform from base_link to third_link (a 1.57 rad yaw rotation, with no translation). RViz then has the TF data it needs to place the link.